



南开大学
Nankai University

南 开 大 学

计 算 机 学 院

数据安全实验报告

秘密共享实践

姓名：姚知言

年级：2022 级

专业：计算机科学与技术

指导教师：刘哲理

2025 年 4 月 23 日

目录

一、 实验要求 1

二、 实验环境 1

三、 实验过程与解释 1

 (一) 复现实验 4-2 及原理解释 1

 1. 环境准备 1

 2. ss_function.py 实现 1

 3. ss_student.py 实现 2

 4. ss_count_student.py 实现 4

 5. vote_counter.py 实现 5

 (二) 改进：实现三个人秘密平均值的计算 5

 1. function.py 5

 2. secret.py 7

 3. count.py 8

 4. avg.py 9

四、 总结与感想 9

一、 实验要求

借鉴实验 4-2，实现三个人对于他们拥有的数据的平均值的计算。

二、 实验环境

实验平台：Linux 系统，Ubuntu 22.04 虚拟机。

高级语言：Python 3.10.12。

三、 实验过程与解释

(一) 复现实验 4-2 及原理解释

实验 4-2 假设有三个同学需要对班里的优秀干部进行投票，最后统计各个班干部获得的票数。

利用 Shamir 秘密共享将各个投票方的投票分享出去并进行隐私求和计算。基于 (2,3) 门限的 Shamir 门限秘密共享进行协议设计。

1. 环境准备

新建 vote 文件夹，在其中建立以下文件。

```
~/Desktop/vote > touch ss_function.py 15:02:22
~/Desktop/vote > touch ss_student.py 15:02:41
~/Desktop/vote > touch ss_count_student.py 15:02:52
~/Desktop/vote > touch vote_counter.py 15:03:22
```

图 1: 新建文件

2. ss_function.py 实现

该文件实现了多项式的生成、计算和重构功能的辅助函数，用于秘密共享方案中的多项式操作。

- quickpower(a, b, p): 在 $O(\log n)$ 的复杂度下高效计算 a 的 b 次方对 p 取模的结果。
- get_polynomial(x0, T, p, fname): 根据常数项系数（秘密），次数和模数，生成多项式，返回多项式参数列表。
- count_polynomial(f, x, p): 多项式定点求值。
- restructure_polynomial(x, fx, t, p): 多项式插值法计算 $f(0)$ ，即秘密值。

```
1 import random
2 #快速幂计算  $a^b \pmod p$ 
3 def quickpower(a,b,p):
4     a=a%p
5     ans=1
6     while b!=0:
7         if b&1:
8             ans=(ans*a)%p
```

```

9         b>>=1
10        a=(a*a)%p
11        return ans
12
13    #构建多项式: x0为常数项系数, T为最高次项次数, p为模数, fname为多项式名
14    def get_polynomial(x0,T,p,fname):
15        f=[]
16        f.append(x0)
17        for i in range(0,T):
18            f.append(random.randrange(0,p))
19        #输出多项式
20        f_print='f'+fname+'='+str(f[0])
21        for i in range(1,T+1):
22            f_print+=' '+str(f[i])+'x^'+str(i)
23        print(f_print)
24        return f
25    #计算多项式值
26    def count_polynomial(f,x,p):
27        ans=f[0]
28        for i in range(1,len(f)):
29            ans=(ans+f[i]*quickpower(x,i,p))%p
30        return ans
31
32    #重构函数f并返回f(0)
33    def restructure_polynomial(x,fx,t,p):
34        ans=0
35        #利用多项式插值法计算出x=0时多项式的值
36        for i in range(0,t):
37            fx[i]=fx[i]%p
38            fxi=1
39            #在模p下, (a/b)%p=(a*c)%p, 其中c为b在模p下的逆元, c=b^(p-2)%p
40            for j in range(0,t):
41                if j !=i:
42                    fxi=(-1*fxi*x[j]*quickpower(x[i]-x[j],p-2,p))%p
43            fxi=(fxi*fx[i])%p
44            ans=(ans+fxi)%p
45        return ans

```

3. ss_student.py 实现

每个学生输入自己的投票值 s , 将 s 作为常数项, 生成一个随机的一次多项式 $f(x) = s + ax$ 。计算该多项式在 $x=1,2,3$ 处的值, 得到三个点 $(1,f(1)), (2,f(2)), (3,f(3))$ 。将这三个点分别保存到文件中, 供其他学生使用。

```

1 import ss_function as ss_f
2 #设置模数p
3 p=1000000007
4 print(f'模数p: {p}')

```

```

5 #输入参与方id以及秘密s
6 id=int(input("请输入参与方id:"))
7 s=int(input(f'请输入student_{id}的投票值s:'))
8
9 #秘密份额为(share_x,share_y)
10 shares_x=[1,2,3]
11 shares_y=[]
12
13 #计算多项式及秘密份额(t=2,n=3)
14 print(f'Student_{id}的投票值的多项式及秘密份额: ')
15 f=ss_f.get_polynomial(s,1,p,str(id))
16 temp=[]
17 for j in range(0,3):
18     temp.append(ss_f.count_polynomial(f,shares_x[j],p))
19     print(f'({shares_x[j]},{temp[j]})')
20     shares_y.append(temp[j])
21
22 #Student_id将自己的投票值的秘密份额分享给另外两个学生
23 #将三份秘密份额分别保存到student_id_1.txt,student_id_2.txt,student_id_3.txt
24 #Student_i获得Student_id_i.txt
25 for i in range(1,4):
26     with open(f'student_{id}_{i}.txt','w') as f:
27         f.write(str(shares_y[i-1]))

```

执行 3 次该程序，分别输入 3 位同学的参与方 id 和投票结果。执行结果如图2所示。

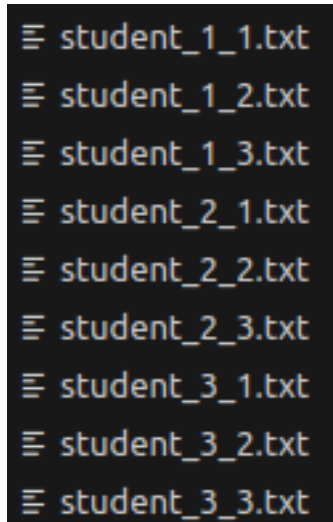
```

~/Desktop/vote > python3 ss_student.py 15:58:55
模数p: 1000000007
请输入参与方id:1
请输入student_1的投票值s:0
Student_1的投票值的多项式及秘密份额:
f1=0+594181586x^1
(1,594181586)
(2,188363165)
(3,782544751)
~/Desktop/vote > python3 ss_student.py 6s 15:59:38
模数p: 1000000007
请输入参与方id:2
请输入student_2的投票值s:1
Student_2的投票值的多项式及秘密份额:
f2=1+735339215x^1
(1,735339216)
(2,470678424)
(3,206017632)
~/Desktop/vote > python3 ss_student.py 5s 15:59:46
模数p: 1000000007
请输入参与方id:3
请输入student_3的投票值s:0
Student_3的投票值的多项式及秘密份额:
f3=0+551613746x^1
(1,551613746)
(2,103227485)
(3,654841231)

```

图 2: 投票秘密值共享

执行完毕后，秘密份额被保存在 txt 文档中，如图3所示。



```

≡ student_1_1.txt
≡ student_1_2.txt
≡ student_1_3.txt
≡ student_2_1.txt
≡ student_2_2.txt
≡ student_2_3.txt
≡ student_3_1.txt
≡ student_3_2.txt
≡ student_3_3.txt

```

图 3: 保存的 txt 文档

比如说, 学生 1 的结果被保存在 “student_1_1”, “student_1_2”, “student_1_3” 中。

4. ss_count_student.py 实现

每个学生收集其他学生分享给自己的秘密份额, 将三个秘密份额相加, 得到 $d_i = a_i + b_i + c_i$ ($i=1,2,3$), 保存这个求和结果。

这个过程本质上是多项式结果的相加。如果 $f_a(0)=a, f_b(0)=b, f_c(0)=c$, 那么 $f_a(x)+f_b(x)+f_c(x)$ 在 $x=0$ 处的值就是 $a+b+c$ 。

```

1 p=1000000007
2 #输入参与方id
3 id=int(input("请输入参与方id:"))
4 #Student_id读取属于自己的秘密份额student_1_id.txt, student_2_id.txt,
   student_3_id.txt
5 data=[]
6 for i in range(1,4):
7     with open(f'student_{i}_{id}.txt', "r") as f: #打开文本
8         data.append(int(f.read())) #读取文本
9 #计算三个秘密份额的和
10 d=0
11 for i in range(0,3):
12     d=(d+data[i])%p
13 #将求和后的秘密份额保存到文件d_id.txt内
14 with open(f'd_{id}.txt','w') as f:
15     f.write(str(d))

```

3 位同学分别执行此过程, 得到结果并保存, 如图4所示。

```
~/Desktop/vote > python3 ss_count_student.py
请输入参与方id:1
~/Desktop/vote > python3 ss_count_student.py
请输入参与方id:2
~/Desktop/vote > python3 ss_count_student.py
请输入参与方id:3
```

图 4: 执行过程

保存的结果如图5所示。

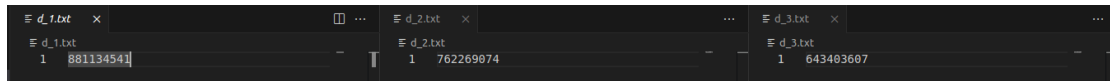


图 5: 加法同态保存结果

比如说, 学生 1 取出 “student_1_1”, “student_2_1”, “student_3_1” 中的结果, 相加后存入 “d_1” 中。

5. vote_counter.py 实现

读取任意两个学生的求和结果 (此处指定学生 2 和 3), 使用拉格朗日插值恢复总票数。

```
1 import ss_function as ss_f
2 #设置模数p
3 p=1000000007
4 #随机选取两个参与方, 例如student2和student3, 获得d2,d3, 从而恢复出d=a+b+c
5 #读取d2,d3
6 d_23=[]
7 for i in range(2,4):
8     with open(f'd_{i}.txt', "r") as f: #打开文本
9         d_23.append(int(f.read())) #读取文本
10 #加法重构获得d
11 d=ss_f.restructure_polynomial([2,3],d_23,2,p)
12 print(f'得票结果为: {d}')
```

运行结果如图6所示, 正确。

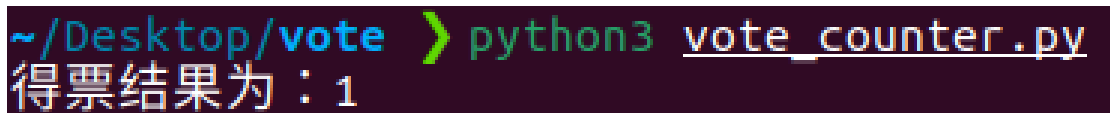


图 6: 运行结果

(二) 改进: 实现三个人秘密平均值的计算

1. function.py

相比于 ss_function.py, 由于在现在的计算中需要用到除法, 主要增加了模逆元的运算。

```
1 import random
2 #快速幂计算 a^b%p
3 def quickpower(a,b,p):
```

```

4     a=a%p
5     ans=1
6     while b!=0:
7         if b&1:
8             ans=(ans*a)%p
9             b>>=1
10            a=(a*a)%p
11    return ans
12
13    #构建多项式: x0为常数项系数, T为最高次项次数, p为模数, fname为多项式名
14    def get_polynomial(x0,T,p,fname):
15        f=[]
16        f.append(x0)
17        for i in range(0,T):
18            f.append(random.randrange(0,p))
19        #输出多项式
20        f_print='f '+fname+'='+str(f[0])
21        for i in range(1,T+1):
22            f_print+=' '+str(f[i])+'x^'+str(i)
23        print(f_print)
24        return f
25    #计算多项式值
26    def count_polynomial(f,x,p):
27        ans=f[0]
28        for i in range(1,len(f)):
29            ans=(ans+f[i]*quickpower(x,i,p))%p
30        return ans
31
32    #重构函数f并返回f(0)
33    def restructure_polynomial(x,fx,t,p):
34        ans=0
35        #利用多项式插值法计算出x=0时多项式的值
36        for i in range(0,t):
37            fx[i]=fx[i]%p
38            fxi=1
39            #在模p下, (a/b)%p=(a*c)%p, 其中c为b在模p下的逆元, c=b^(p-2)%p
40            for j in range(0,t):
41                if j !=i:
42                    fxi=(-1*fxi*x[j]*quickpower(x[i]-x[j],p-2,p))%p
43            fxi=(fxi*fx[i])%p
44            ans=(ans+fxi)%p
45        return ans
46
47    #模逆元运算
48    def modinv(a, p):
49        return quickpower(a, p-2, p)

```


2. secret.py

基本参照 ss_student.py 实现，调整了输入输出和注释。

```
1 import function as func
2 #设置模数p
3 p=1000000007
4 print(f'模数p:{p}')
5 #输入参与方id以及秘密s
6 id=int(input("请输入参与方id:"))
7 s=int(input(f'请输入用户{id}的秘密值s:'))
8
9 #秘密份额为(share_x,share_y)
10 shares_x=[1,2,3]
11 shares_y=[]
12
13 #计算多项式及秘密份额(t=2,n=3)
14 print(f'用户{id}的秘密值的多项式及秘密份额:')
15 f=func.get_polynomial(s,1,p,str(id))
16 temp=[]
17 for j in range(0,3):
18     temp.append(func.count_polynomial(f,shares_x[j],p))
19     print(f'({shares_x[j]},{temp[j]})')
20     shares_y.append(temp[j])
21
22 #秘密的秘密份额分享给另外两个用户
23 #将三份秘密份额分别保存到user_id_1.txt,user_id_2.txt,user_id_3.txt
24 #user_i获得user_id_i.txt
25 for i in range(1,4):
26     with open(f'user_{id}_{i}.txt','w') as f:
27         f.write(str(shares_y[i-1]))
```

运行结果如图7所示。三个用户的秘密值我在这里分别设定为我的学号 2211290，当前日期 20250423，以及当前时间 112939。

```

~/Desktop/secret > python3 secret.py 11:29:10
模数p:1000000007
请输入参与方id:1
请输入用户1的秘密值s:2211290
用户1的秘密值的多项式及秘密份额:
f1=2211290+469142951x^1
(1,471354241)
(2,940497192)
(3,409640136)
~/Desktop/secret > python3 secret.py 4s 11:29:19
模数p:1000000007
请输入参与方id:2
请输入用户2的秘密值s:20250423
用户2的秘密值的多项式及秘密份额:
f2=20250423+247434094x^1
(1,267684517)
(2,515118611)
(3,762552705)
~/Desktop/secret > python3 secret.py 14s 11:29:39
模数p:1000000007
请输入参与方id:3
请输入用户3的秘密值s:112939
用户3的秘密值的多项式及秘密份额:
f3=112939+819698457x^1
(1,819811396)
(2,639509846)
(3,459208296)

```

图 7: secret.py 运行结果

3. count.py

基本参照 ss_count_student.py 实现，调整了输入输出文字。同时由于先前的设计中，没有直接输出这一步结果的打印，而是直接存在文件中，在这里增加了这部分打印。

```

1 p=1000000007
2 #输入参与方id
3 id=int(input("请输入参与方id:"))
4 #user读取属于自己的秘密份额user_1_id.txt,user_2_id.txt,user_3_id.txt
5 data=[]
6 for i in range(1,4):
7     with open(f'user_{i}_{id}.txt', "r") as f: #打开文本
8         data.append(int(f.read())) #读取文本
9 #计算三个秘密份额的和
10 d=0
11 for i in range(0,3):
12     d=(d+data[i])%p
13 print(f'x={id}求得的秘密份额和为{d}')
14 #将求和后的秘密份额保存到文件d_id.txt内
15 with open(f'd_{id}.txt','w') as f:
16     f.write(str(d))

```

运算结果如图8所示。

```
~/Desktop/secret > python3 count.py 11:30:58
请输入参与方id:1
x=1求得的秘密份额和为558850147
~/Desktop/secret > python3 count.py 11:31:42
请输入参与方id:2
x=2求得的秘密份额和为95125635
~/Desktop/secret > python3 count.py 11:31:46
请输入参与方id:3
x=3求得的秘密份额和为631401130
```

图 8: count.py 运算结果

4. avg.py

相比于 vote_counter.py 增加了平均值计算，并修改了打印输出。

平均值计算使用 2 种设计，一种是直接进行的除法运算，可以保留非整除结果的小数部分。另一种是通过乘以模逆元进行的除法运算，便于保证结果是整数以参与后续计算，但是会舍弃小数部分。

```
1 import function as func
2 #设置模数p
3 p=1000000007
4 #随机选取两个参与方，例如student2和student3，获得d2,d3，从而恢复出d=a+b+c
5 #读取d2,d3
6 d_23=[]
7 for i in range(2,4):
8     with open(f'd_{i}.txt', "r") as f: #打开文本
9         d_23.append(int(f.read())) #读取文本
10 #加法重构获得d
11 d=func.restructure_polynomial([2,3],d_23,2,p)
12
13 #直接除法
14 print(f'平均值(除法)结果为:{d/3.0}')
15
16 #逆元运算
17 inv_3=func.modinv(3,p)
18 print(f'3在模{p}运算下的逆元是:{inv_3}')
19 print(f'平均值(逆元运算)结果为:{(d*inv_3)%p}')
```

运算结果如图9所示。运算结果正确， $(2211290+20250423+112939)/3 = 7524884$ 。

```
~/Desktop/secret > python3 avg.py 11:31:50
平均值(除法)结果为:7524884.0
3在模1000000007运算下的逆元是:333333336
平均值(逆元运算)结果为:7524884
```

图 9: avg.py 运算结果

四、 总结与感想

在本次实验中，我与课堂理论结合，对 Shamir 秘密共享的方法更加熟悉，也加深了对快速幂算法，多项式插值法等方法的研究。

此外，在本次实验中，我也加深了数论中逆元计算等概念的理解。

NIKU